

Ressource R2.02 : Développement d'application avec IHM

JavaFX - Séance TP 4 : liaison de données (Databinding)

1 – Présentation générale

Un problème récurrent dans les interfaces graphiques est celui de la synchronisation du modèle (représentation des objets métier) et de l'interface : comment faire pour que les valeurs affichées dans l'interface correspondent toujours à celles de l'objet ? Comment faire pour que toute saisie soit immédiatement appliquée à l'objet ?

En JavaFX, un mécanisme permet de lier une donnée à une autre si bien que toute modification de la donnée est répercutée automatiquement dans la donnée qui lui est liée. Ce mécanisme permet même de lier des données entre elles de manière symétrique (ou bidirectionnelle).

Dans ce schéma, *donnée2* est liée à *donnée1* : donc toute modification de *donnée1* entraîne la modification de *donnée2*.



En javaFX, de telles données sont appelées des **propriétés** (*properties* en anglais).

2 - Les propriétés

À l'instar d'une variable, une propriété en JavaFX permet de stocker une valeur. Mais cette propriété est "observable", c'est-à-dire qu'une autre propriété peut l'observer et donc savoir si sa valeur change.

L'interface *Property*, et les classes qui l'implémentent (*IntegerProperty*, *FloatProperty*, *StringProperty*, etc.) permettent l'utilisation et la liaison de propriétés.

La valeur d'une propriété est accessible par `get ()` et elle est modifiable par `set ()`.

2.1 – Liaison simple

Prenons l'exemple où la propriété *p1* contient un entier et où *p2* est une propriété liée à *p1*.



La méthode **bind()** permet de lier *p2* à *p1*.

Exercice 1

Tapez et exécutez ce code :

```
public class Exemple {
    public static void main(String[] args) {

        IntegerProperty p1 = new SimpleIntegerProperty(100);
        IntegerProperty p2 = new SimpleIntegerProperty();

        // p2 est liée à p1
        p2.bind(p1);

        // toute modification de p1 entraîne la modification de p2
        System.out.println(p2.get());
        p1.setValue(200.0);
        System.out.println(p2.get());

    }
}
```

C'est ici que le lien est établi entre les deux propriétés.

Remarques :

- la classe `IntegerProperty` est une classe abstraite : la classe `SimpleIntegerProperty` en hérite et elle est concrète.
- dans ce cas de liaison simple, la valeur de *p2* ne peut pas être modifiée directement ; `p2.set(...)` provoquera une erreur du style *"a bound value cannot be set"*.

2.2 – Liaison bidirectionnelle

Dans une liaison bidirectionnelle, ça marche dans les deux sens : si deux propriétés *p1* et *p2* sont liées de manière bidirectionnelle, alors quand *p1* est modifiée, *p2* l'est aussi et réciroquement. Il faut pour cela utiliser la méthode `bindBidirectional(...)`.



Exercice 2

Modifiez le code précédent comme suit et testez :

```
public class Exemple2 {
    public static void main(String[] args) {

        DoubleProperty p1 = new SimpleDoubleProperty(100);
        DoubleProperty p2 = new SimpleDoubleProperty(150);

        // p1 et p2 sont liées l'une à l'autre
        p2.bindBidirectional(p1);

        // toute modification d'une propriété modifie l'autre
        System.out.println(p2.get());
        p1.setValue(200.0);
        System.out.println(p2.get());

        System.out.println(p1.get());
        p2.setValue(250.0);
        System.out.println(p1.get());

    }
}
```

C'est ici que la liaison bidirectionnelle est établie.

2.3 – Liaison complexe

Les liaisons simples vues précédemment assurent une égalité entre les valeurs de deux données : $p1 = p2$.

On parle de liaison complexe lorsqu'une donnée est le résultat d'un calcul basé sur une ou plusieurs autres données. C'est le cas lorsque par exemple on dispose d'une donnée *rayon* et d'une donnée *perimetre* liées par l'expression :

$$\text{perimetre} = 2 \times \Pi \times \text{rayon} \quad (1)$$

ou encore de trois données liées par l'expression :

$$\text{total} = p1 + p2 \quad (2)$$

Les liaisons complexes sont utilisables grâce à l'interface **NumberExpression** qui propose entre autres les méthodes `add(...)`, `multiply(...)`, `subtract(...)` et `divide(...)`. Une variable liée (une "binding") calcule une valeur qui dépend de plusieurs sources appelées ses **dépendances**.

Dans l'expression (1) *perimetre* est une "binding" ou "donnée liée" et *rayon* sa **dépendance**. Ce qui donne en java :

```
IntegerProperty rayon = new SimpleIntegerProperty(10);

DoubleBinding perimetre = rayon.multiply(2*Math.PI);
```

Dans l'expression (2) `total` est une binding et `p1` et `p2` ses dépendances :

```
IntegerProperty p1      = new SimpleIntegerProperty(10);  
IntegerProperty p2      = new SimpleIntegerProperty(20);  
  
NumberBinding total = p1.add(p2) ;
```

Exercice 3

Dans un programme Java, programmez ces deux liaisons complexes (1) et (2) puis affichez le total et le périmètre après avoir modifié l'une de leurs dépendances.

Important : bien sûr, une liaison complexe ne peut pas être bidirectionnelle !

3 – La classe Bindings

On l’a vu précédemment, pour lier deux propriétés p1 et p2 de manière bidirectionnelle, on peut écrire :

```
p2.bindBidirectional(p1);
```

3.1 – Une autre manière de lier des données

La classe Bindings est une classe utilitaire qui permet de définir des liaisons entre données, et notamment des liaisons complexes, de manière plus intuitive. Elle permet aussi de créer des liaisons entre données de type différents. Enfin, cette classe permet de définir des liaisons conditionnelles.

Par exemple, pour lier p1 et p2 de manière bidirectionnelle, on pourra écrire :

```
Bindings.bindBidirectional(p2, p1);
```

La classe Bindings permet de définir plus facilement des liaisons complexes. Par exemple, assurer une liaison de type $res = a*b + c*d$ s’écrit :

```
NumberBinding res = add(multiply(a,b),multiply(c,d));1
```

au lieu de :

```
NumberBinding res = a.multiply(b).add(c.multiply(d));
```

¹ En réalité il faut écrire : `res=Bindings.add(Bindings.multiply(a,b),Bindings.multiply(c,d));`

3.2 – Conversion de type dans une liaison

Il est parfois utile de créer une liaison entre données de type différent, et la classe Bindings le permet.

3.2.1 – *Liaison bidirectionnelle*

Pour une liaison bidirectionnelle, on utilise un "converter". Par exemple pour lier ensemble une propriété de type entier et une propriété de type chaîne :

```
SimpleIntegerProperty nb      = new SimpleIntegerProperty();
SimpleStringProperty  texte = new SimpleStringProperty();

// création du "converter" nombre <-> chaîne
StringConverter<Number> converter = new NumberStringConverter();

// liaison bidirectionnelle qui utilise le "converter"
Bindings.bindBidirectional(texte, nb, converter);
```

Exercice 4

Tapez ce code et programmez l’affichage des deux propriétés après modification de l’une puis de l’autre.

3.2.2 – *Liaison simple*

Dans le cas d'une liaison simple, il faut programmer explicitement la conversion en créant une liaison supplémentaire par une méthode du style `createXXXBinding`, où `XXX` représente le type de la donnée liée (`String`, `Integer`, `Float`, etc.).

La syntaxe de ces méthodes est :

```
createXXXBinding( <lambda expression>, <dépendance> )
```

Représente la fonction de conversion. Une expression lambda est du style : `param->code`

Représente la donnée à lier

Cet exemple montre comment lier la propriété `texte` à la propriété `nb` :

```
SimpleIntegerProperty nb      = new SimpleIntegerProperty();
SimpleStringProperty  texte = new SimpleStringProperty();

// création d'une liaison : convert est liée à nb moyennant une conversion
// de type : la valeur de nb est transformée en chaîne
StringBinding convert =
    Bindings.createStringBinding( () -> String.valueOf(nb.get()), nb );

// liaison avec cette nouvelle "Binding"
texte.bind(convert);
```

Ici, l'expression lambda n'a pas de paramètre et retourne la valeur de `nb` sous forme de `String`.

Exercice 5

Tapez ce code et programmez l'affichage des deux propriétés après modification de l'une puis de l'autre.

Vous constaterez que vous ne pouvez pas modifier la valeur de la propriété liée `texte`.

3.3 – Liaison conditionnelle

Un autre avantage de la classe Bindings est la possibilité de créer des liaisons conditionnelles. Dans une telle liaison, la valeur d'une propriété correspond toujours à la valeur d'une autre propriété observable, mais cette liaison est soumise à une condition.

Pour cela il faut utiliser les méthodes `when()`, `then()` et `otherwise()` :

```
Bindings.when(<condition observable>).then(<val>).otherwise(<val>)
```

Par exemple, pour définir une liaison complexe réalisant : $\text{moyenne} = \text{somme} / \text{nb}$ il faut prévoir le cas où nb vaut 0 (et dans ce cas prévoir -1 par exemple comme résultat).

```
SimpleDoubleProperty somme    = new SimpleDoubleProperty(89);
SimpleDoubleProperty nb      = new SimpleDoubleProperty(8);
SimpleDoubleProperty moyenne = new SimpleDoubleProperty();

BooleanBinding casErreur = Bindings.equal(nb, 0);

moyenne.bind(Bindings.when(casErreur).then
              (-1.0).otherwise(Bindings.divide(somme, nb)));
```

Ici, la propriété `moyenne` est liée à `somme/nb` sauf en cas d'erreur où elle vaudra -1.

Exercice 6

Tapez ce code et programmez l'affichage des deux propriétés après modification de l'une puis de l'autre.

Vous constaterez que vous ne pouvez pas modifier la valeur de la propriété liée `moyenne`.

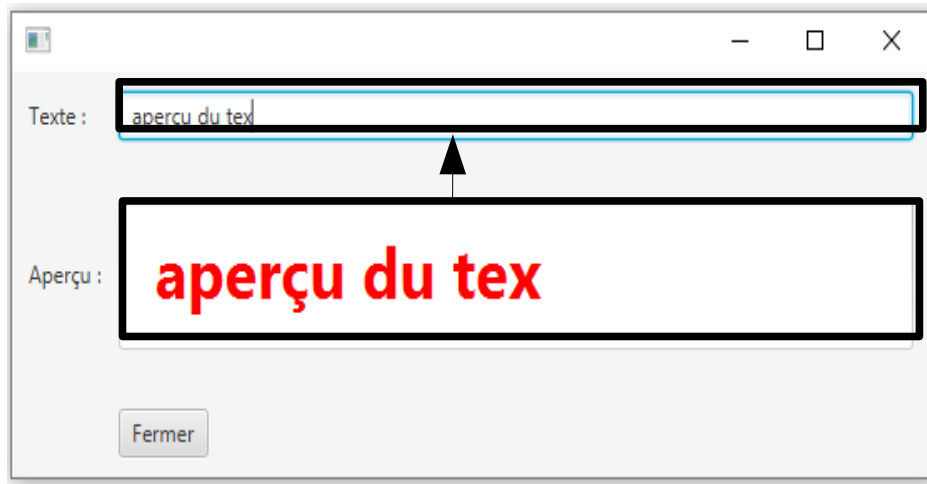
4 – Liaison de données dans les IHM

En JavaFX, les caractéristiques d'un composant graphique sont des propriétés et sont donc observables. Par exemple pour un `TextField`, on trouve les propriétés :

`textProperty`, `editableProperty`, `disableProperty`, `effectProperty`, etc.

4.1 - Liaison simple dans une IHM

On peut par exemple créer un lien entre deux propriétés de composants, comme dans la classe **FenTextField** où la propriété "texte" du deuxième `TextField` est liée à celle du premier `TextField`.



Extrait du code :

```
Label lblTexte1 = new Label("Texte :");
TextField saisie = new TextField();
Label lblTexte2 = new Label("Aperçu :");
TextField apercu = new TextField();

apercu.textProperty().bind(saisie.textProperty());
```

Établissement de la liaison schématisée ci-dessus.

Exercice 7

Récupérez le fichier **FenTextField.java** sur Moodle et exécutez la classe. Vous constaterez que le deuxième champ n'est pas modifiable.

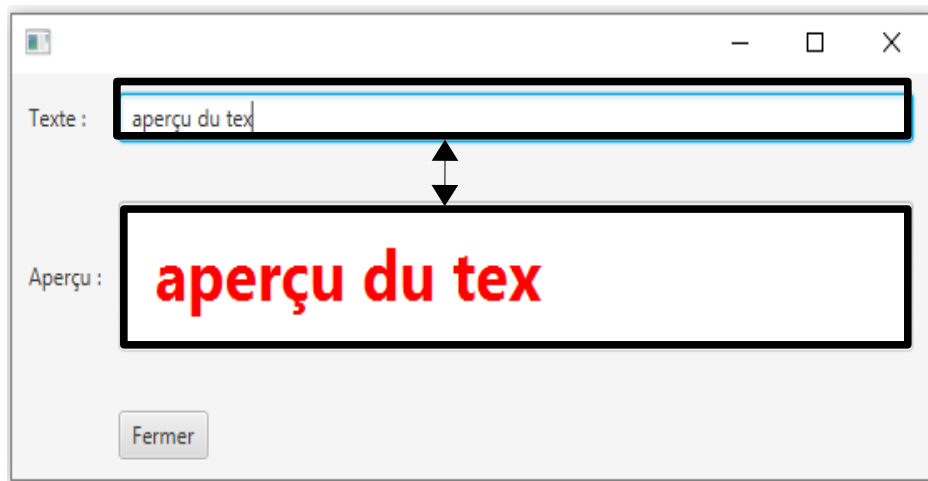
Exercice 8



Complétez la classe **FenTitre** (le fichier java est disponible sur Moodle) de manière à ce que le titre de la fenêtre s'actualise automatiquement au fur et à mesure de la frappe dans le TextField. Autrement dit, faites en sorte que le titre de la fenêtre soit lié à la propriété `textProperty` du TextField.

4.2 – Liaison bidirectionnelle dans une IHM

Reprenons l'exemple précédent de la page 9 (FenTextField) mais en établissant cette fois une liaison bidirectionnelle.



Extrait du code :

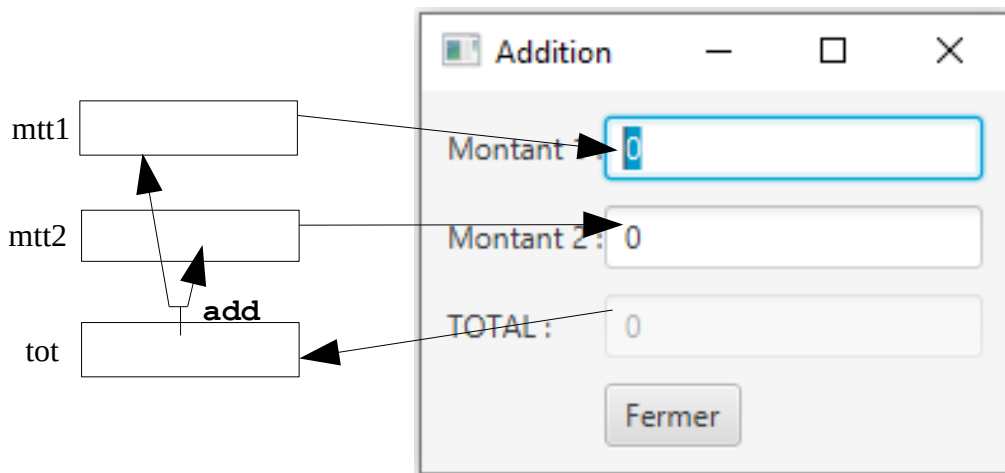
```
Label lblTexte1 = new Label("Texte :");  
TextField saisie = new TextField();  
Label lblTexte2 = new Label("Aperçu :");  
TextField apercu = new TextField();  
  
apercu.textProperty().bindBidirectional(saisie.textProperty());
```

Établissement du lien
bidirectionnel.

Vous constaterez cette fois que les deux TextField sont utilisables et "fonctionnent" de manière symétrique.

4.3 – Liaison complexe

Voici un exemple de liaison complexe entre propriétés où chacune est liée à un champ de formulaire. Toutes ces liaisons sont schématisées ci-dessous :



La saisie d'une valeur dans le champ "Montant 1" ou dans le champ "Montant 2" déclenche une actualisation en chaine jusqu'au champ TOTAL. Remarquez qu'il ne s'agit pas de liaisons bidirectionnelles.

Voici le code associé :

```
// les propriétés
SimpleIntegerProperty mtt1 = new SimpleIntegerProperty();
SimpleIntegerProperty mtt2 = new SimpleIntegerProperty();
SimpleIntegerProperty tt    = new SimpleIntegerProperty();

// les quatre liaisons
tt.bind(mtt1.add(mtt2));

mtt1.bind(Bindings.createDoubleBinding( () ->
Double.parseDouble(montant1.getText()),montant1.textProperty()
));

mtt2.bind(Bindings.createDoubleBinding( () ->
Double.parseDouble(montant2.getText()),montant2.textProperty()
));

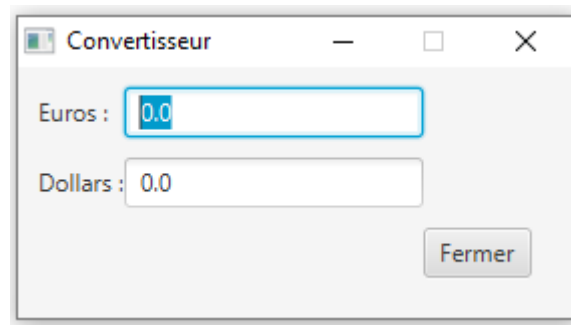
total.textProperty().bind(Bindings.createStringBinding( () ->
String.valueOf(tt.get()), tt));
```

Remarquez la nécessité de convertir les valeurs dans une liaison entre le texte d'un champ de saisie et une propriété numérique.

Le fichier FenAddition.java est disponible sur Moodle.

Exercice 9

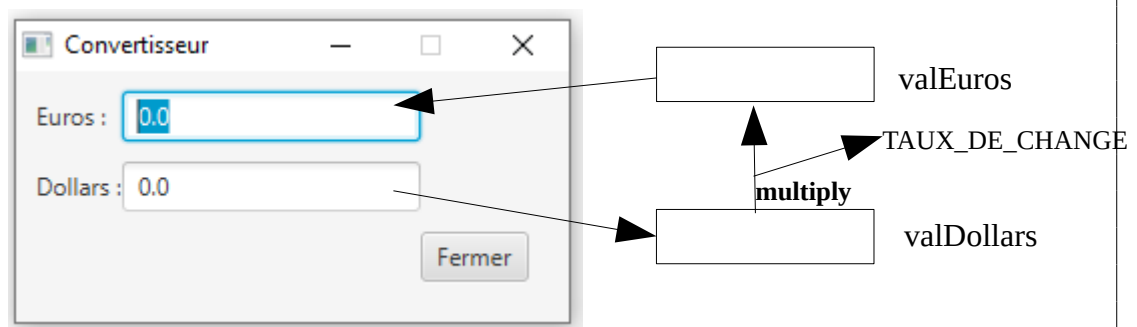
Complétez la classe **FenConvertisseur** disponible sur Moodle de manière à programmer ce convertisseur Euros → Dollar.



La zone Dollars est non éditable. Chaque modification de la valeur en euros doit actualiser le champ Dollars. Le taux de change est déclaré en constante.

Question 1

Dans une première version, vous utiliserez des propriétés intermédiaires pour effectuer le calcul, comme montré dans ce schéma :



Question 2

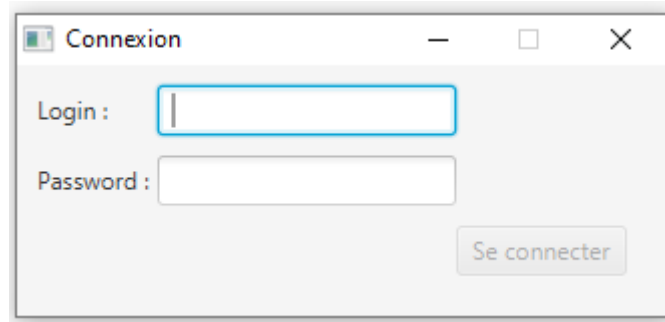
Écrivez une deuxième version en vous passant des deux propriétés intermédiaires.

Remarque : une exception se produit (et pour cause) si à l'exécution on efface complètement la valeur en euros.

4.4 – Liaison conditionnelle : utilisation des méthodes **when**, **then** et **otherwise**

Une utilisation possible de liaison conditionnelle dans les IHM est l'activation/désactivation d'un bouton. On le sait, un bouton doit être désactivé (on dit aussi "grisé") si l'action qu'il représente n'est pas déclenchable. On rencontre souvent cette situation dans les formulaires où le bouton de validation doit être "grisé" si l'un des champs obligatoires n'est pas rempli.

Dans cette fenêtre de connexion, le bouton doit être grisé si le champ *Login* est vide.



Pour cela il faut lier la propriété `disableProperty` du bouton à la valeur `true` ou bien `false` selon une condition :

```
bnSeConnecter.disableProperty().bind(  
    Bindings.when(txtLogin.textProperty().isEmpty())  
        .then(true).otherwise(false)  
);
```

Ce lien assure que l'activation/désactivation du bouton sera fonction de la présence ou non de la valeur du login.

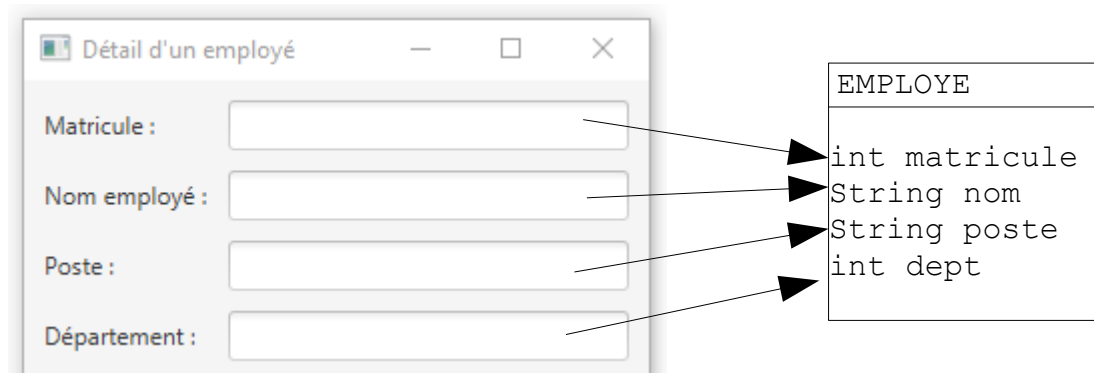
Exercice 10

Récupérez le fichier *FenConnexion.java* depuis Moodle. En fait les champs *Login* et *Password* sont obligatoires tous les deux. Faites en sorte que le bouton *Se Connecter* ne soit actif que lorsque les deux champs sont remplis (et inactif sinon).

Conseil : créez au préalable une `BooleanBinding` pour la condition du `when`.

5 – Synchroniser Modèle et Interface

Il est légitime de vouloir assurer une cohérence automatique entre un objet et sa visualisation dans une IHM. L'idée est alors de lier chaque composant graphique de l'IHM à un attribut de l'objet. Cet exemple montre de telles liaisons et, si un attribut change de valeur, le contenu du `TextField` associé s'actualisera automatiquement.



Traditionnellement, on programme une telle classe `EMPLOYEE` en créant quatre variables d'instance :

```
public class Employe {  
    private int matricule;  
    private String nom;  
    private String poste;  
    private int dept;  
  
    // avec constructeur, getters et setters...  
}
```

Mais cette programmation ne convient plus car pour assurer les liaisons avec les composants de l'IHM, les attributs doivent être **observables**.

5.1 – Les attributs implémentés comme propriétés

On va donc programmer la classe d'une nouvelle manière. Les attributs vont donner lieu à des *propriétés* (et non plus de simples variables) et la classe Java possédera trois accesseurs par attribut : un getter et un setter pour la valeur et un getter pour la propriété.

```
public class Employe {
    private final IntegerProperty matricule;
    private final StringProperty nom;
    private final StringProperty poste;
    private final IntegerProperty dept;

    public Employe(int matricule, String nom, String poste, int dept) {
        this.matricule = new SimpleIntegerProperty(matricule);
        this.nom = new SimpleStringProperty(nom);
        this.poste = new SimpleStringProperty(poste);
        this.dept = new SimpleIntegerProperty(dept);
    }

    public int getMatricule() {
        return matricule.get();
    }
    public void setMatricule(int matricule) {
        this.matricule.set(matricule);
    }
    public IntegerProperty matriculeProperty() {
        return this.matricule;
    }

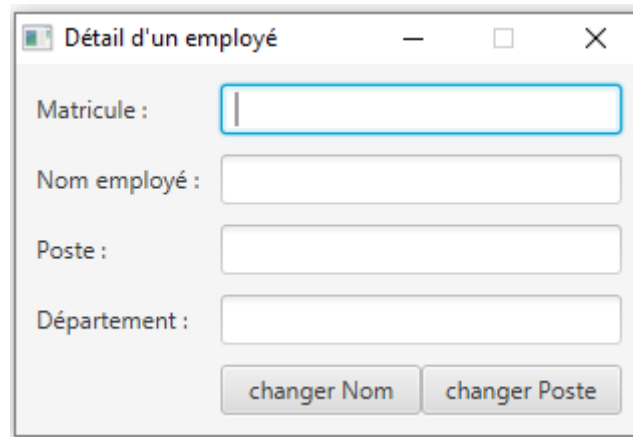
    public String getNom() {
        return nom.get();
    }
    public void setNom(String nom) {
        this.nom.set(nom);
    }
    public StringProperty nomProperty() {
        return this.nom;
    }
    etc.
}
```

déclaration des 4 propriétés observables (remarquez le mot final)

les 3 accesseurs pour la propriété matricule : remarquez bien la syntaxe

Exercice 11

Récupérez les fichiers *Employe.java* et *FenVueSimpleEmploye.java* depuis Moodle.



The screenshot shows a Java Swing window titled "Détail d'un employé". It has a standard title bar with a close button (X). Inside the window, there are four text input fields arranged vertically, each with a label to its left: "Matricule :", "Nom employé :", "Poste :", and "Département :". The "Matricule" field is currently selected, indicated by a blue border and a vertical cursor. Below these fields, there are two buttons: "changer Nom" and "changer Poste".

Complétez la classe *FenVueSimpleEmploye* en y ajoutant les liaisons entre chaque champ de l'IHM et l'attribut correspondant de la classe *Employe*.

Attention, pour les champs *Matricule* et *Département*, il faudra penser à faire une conversion de type.

5.2 – Les collections observables

JavaFX propose une classe utilitaire `FXCollections` qui reprend toutes les méthodes proposées dans la classe `Collections` de Java, à la différence qu'il s'agit de collections observables (`observableList`, `observableHashMap`, `observableSet`, ...).

Un composant graphique lié à une telle collection observable peut donc être automatiquement mis à jour lors de la modification du contenu de la collection (ajout, modification ou suppression d'éléments).

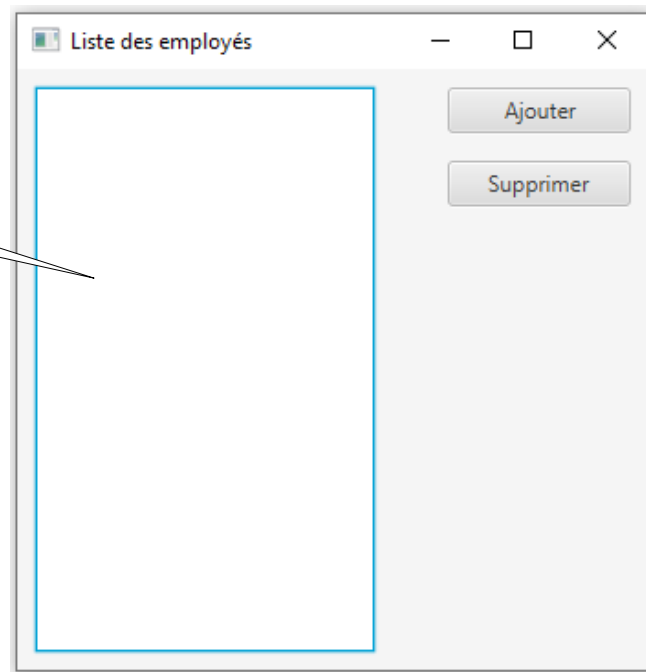
5.2.1 – Exemple avec un `ListView`

Dans l'exemple qui suit, le programme gère une liste observable de noms d'employés déclarée comme ceci :

```
ObservableList<String> liste = FXCollections.observableArrayList();
```

La fenêtre dispose d'un composant `ListView` qui doit être associé à cette liste observable.

Le `ListView` que vous devez lier à la collection.



Exercice 12

Complétez la classe `FenListeEmployes` (disponible sur Moodle), et liez le `ListView` à la liste observable de la manière suivante :

```
listeEmployes.setItems(liste);
```

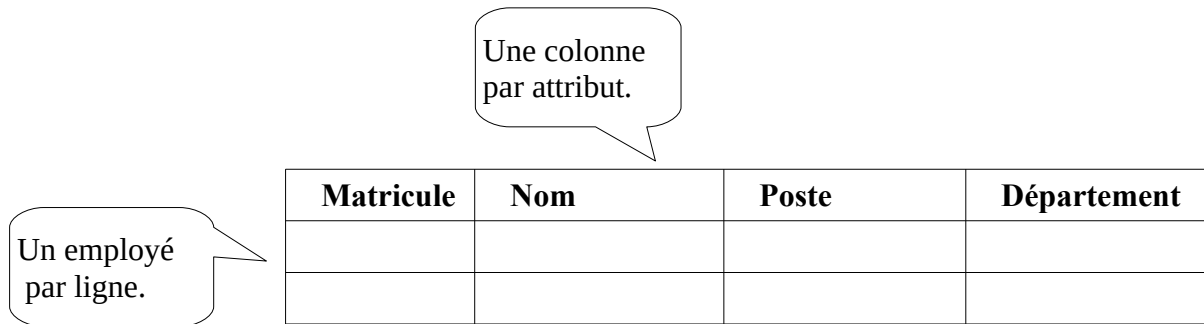
En regardant comment les boutons sont programmés, vous constaterez que tout ajout ou suppression dans la liste se répercute automatiquement sur le `ListView`.

Attention, cliquer sur le bouton *Supprimer* alors que rien n'est sélectionné déclenche une exception. Résolvez ce problème.

5.2.2 – Exemple avec un TableView

Un TableView est un composant qui permet une visualisation tabulaire de données. Il est adapté à la présentation d'une collection d'objets, chaque colonne correspondant à un attribut.

Le TableView est très pratique ici pour afficher une liste d'employés. Chaque colonne possède un entête et contient les valeurs d'un attribut.



Matricule	Nom	Poste	Département

Détaillons maintenant la création d'un TableView. Il faut tout d'abord déclarer et initialiser la variable. Il s'agit ici d'un TableView d'employés :

```
private TableView<Employe> tableEmployes= new TableView<Employe>();
```

Il faut ensuite lui ajouter ses 4 colonnes, une par une. L'ajout d'une colonne se fait en trois temps.

1. Création de la colonne, où on indique l'entête :

```
TableColumn<Employe,Integer> colonne1 =  
    new TableColumn<Employe,Integer>("Matricule");
```

2. Liaison entre la colonne et l'attribut :

```
colonne1.setCellValueFactory(  
    new PropertyValueFactory<Employe,Integer>("matricule"));
```

3. Ajout de la colonne au TableView :

```
tableEmployes.getColumns().add(colonne1);
```

Exercice 13

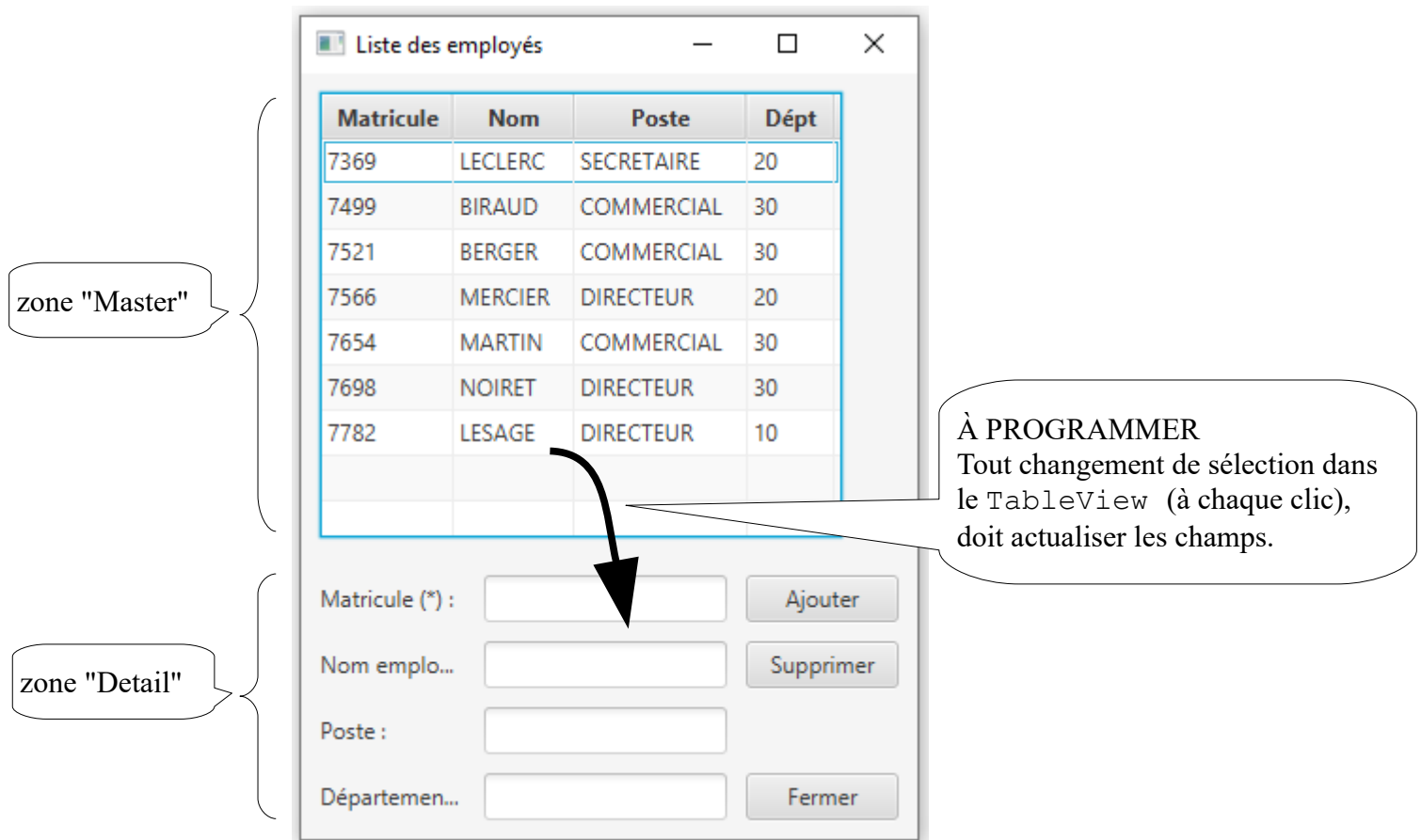
Complétez la classe FenTableEmployes (disponible sur Moodle) : créez et ajoutez les 3 autres colonnes au TableView puis liez le TableView à la liste observable de la manière suivante :

```
tableEmployes.setItems(liste);
```

5.3 – Interface Master-Detail

Une fenêtre ou une page est dite "Master-Detail" lorsqu'elle est structurée en deux zones où une première zone (*Master*) présente une collection d'objets et où une deuxième zone (*Detail*) présente les informations de l'objet sélectionné. Cette structure est fréquemment utilisée dans les IHM.

Dans cette dernière partie, on se propose de créer une fenêtre Master-Detail pour la gestion des employés. Le schéma suivant montre la structure générale de la fenêtre.



Exercice 14

Complétez la classe `FenGestionEmployes` (disponible sur Moodle) :

- 1 – programmer le clic sur le `TableView` (il faut actualiser les 4 champs de la zone "Detail" avec l'employé sélectionné dans la zone "Master"),
- 2 – programmer le clic sur le bouton *Supprimer* (prenez modèle sur la programmation du bouton *Ajouter*. Pensez à effacer les champs après la suppression),
- 3 – griser le bouton *Ajouter* si les données obligatoires ne sont pas toutes saisies,
- 4 – griser le bouton *Supprimer* si aucun employé n'est sélectionné dans le `TableView`.